

## 0 math voodoo

- $e^x \geq 1 + x$
- $\ln(x) \leq x - 1$
- $\log_b a = \frac{1}{\log_a b}$
- $a^{\log_b a} = c^{\log_b a}$
- Stirling's approximation:
$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + O\left(\frac{1}{n}\right)\right)$$

- $\log(n!) \in \Theta(n \log n)$
- Sum of geometric series:

$$\sum_{i=0}^n r^i = \begin{cases} \frac{1-r^{n+1}}{1-r} & r \neq 1 \\ n+1 & r = 1 \end{cases}$$

- Sum of arithmetic series:
$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$
- Sum of first  $n$  integers to the  $k$ -th power (Faulhaber's formula):

$$\sum_{i=1}^n i^k = \frac{n^{k+1}}{k+1} + O(n^k)$$

- Harmonic Number:
$$\sum_{i=1}^n \frac{1}{i} = \ln n + O(1)$$

## 1 Asymptotic Analysis

Algorithmic Analysis involves the **resources needed** by an algorithm:

- Time**: the number of steps taken to complete.
- Space**: the amount of memory used to complete. data fits in memory)

**Running time** is a function of the **size of the input**, denoted as  $n$ , which can be measured in various ways, such as the number of elements in an array, the number of nodes in a graph, etc. Typically, we perform **worst-case analysis** considering the **maximum running time** for any input of size (at most)  $n$ , or **average-case analysis** considering the **expected running time** over all inputs of size  $n$  under some probability distribution.

### RAM Model of Computation

In our model, we analyse algorithms under the assumptions that:

- Each basic unit of memory is a **word**.
- RAM** is an array of words, storing instructions and data.
- One unit of time taken to access any word.
- Constant time** for each operation. (*Except exponentiation*)
- If input or output is too large, time taken depends on the number of words needed to store it.

Running time is **roughly proportional to the number of instructions** executed in the RAM model, independent of the actual machine.

## 1.1 Asymptotic Notation

### Definitions

$f \in O(g)$  if  
 $\exists c, n_0 > 0 \quad \forall n \geq n_0 \quad 0 \leq f(n) \leq c \cdot g(n).$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty \implies f \in O(g).$$

$f$  grows **at most as fast as**  $g$  asymptotically.

$f \in \Omega(g)$  if  
 $\exists c, n_0 > 0 \quad \forall n \geq n_0 \quad 0 \leq c \cdot g(n) \leq f(n).$

$$0 < \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \implies f \in \Omega(g).$$

$f$  grows **at least as fast as**  $g$  asymptotically.

$f \in \Theta(g)$  if  $f \in O(g)$  and  $f \in \Omega(g)$ , i.e.

$\exists c_1, c_2, n_0 > 0, \forall n \geq n_0, 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n).$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c \in (0, \infty) \implies f \in \Theta(g).$$

$f$  grows **as fast as**  $g$  asymptotically.

$f \in o(g)$  if  
 $\forall c > 0 \quad \exists n_0 > 0 \quad \forall n \geq n_0 \quad 0 \leq f(n) < c \cdot g(n).$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \implies f \in o(g).$$

$f$  grows **slower than**  $g$  asymptotically.

$f \in \omega(g)$  if  
 $\forall c > 0 \quad \exists n_0 > 0 \quad \forall n \geq n_0 \quad 0 \leq c \cdot g(n) < f(n).$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty \implies f \in \omega(g).$$

$f$  grows **faster than**  $g$  asymptotically.

### Properties

For functions  $f, g, h$ :

- Reflexivity**:  $f \in O(f), f \in \Omega(f), f \in \Theta(f).$
- Transitivity**: For  $A \in \{O, \Omega, \Theta, o, \omega\}$   
 $f \in A(g)$  and  $g \in A(h) \implies f \in A(h).$
- Symmetry**:  $f \in \Theta(g) \iff g \in \Theta(f).$
- Complementarity**:

$$f \in O(g) \iff g \in \Omega(f)$$

$$f \in o(g) \iff g \in \omega(f)$$

## 2 Recurrences and Master Theorem

Given a **recursive algorithm**  $A$ , we can express its running time as a **recurrence relation**  $T(n)$ , describing its running time in terms of the that of its recursive calls.

### 2.1 Telescoping

**Telescoping** involves **unrolling** the recurrence until we reach the base case, and then summing up the contributions from each level of recursion.

$$\text{e.g. } T(n) = \begin{cases} \Theta(1) & \text{if } n \leq 1 \\ 2T\left(\frac{n}{2}\right) + n & \text{if } n > 1 \end{cases}$$

$$\frac{T(n)}{n} = \frac{T\left(\frac{n}{2}\right)}{\frac{n}{2}} + 1$$

$$\frac{T\left(\frac{n}{2}\right)}{\frac{n}{2}} = \frac{T\left(\frac{n}{4}\right)}{\frac{n}{4}} + 1$$

$\vdots$

$$\frac{T(2)}{2} = \frac{T(1)}{1} + 1$$

$$\implies \frac{T(n)}{n} = \frac{T(1)}{1} + \log n$$

$$= O(\log n)$$

### 2.2 Substitution

**Substitution** requires **guessing** the form of the solution, and then **proving** it by induction.

$$\text{e.g. } T(n) = \begin{cases} c & \text{if } n \leq 1 \\ 4T\left(\lfloor \frac{n}{2} \rfloor\right) + n & \text{if } n > 1 \end{cases}$$

Induction Hypothesis:  $T(k) \leq (c+1)k^2 - n$  for all  $k < n$

$$\text{Induction Step: } T(n) = 4T\left(\lfloor \frac{n}{2} \rfloor\right) + n$$

$$\leq 4(c+1)\left(\frac{n}{2}\right)^2 - 4\left(\frac{n}{2}\right) + n$$

$$= (c+1)n^2 - n$$

## Making good guesses

Consider  $T(n) = \text{Recursive calls} + \text{"Merging" time}$ . Substituting the guess back into the recurrence, and compare to the guess each of the contributions:

- Time contributed by **recursive calls** (Including constants.)
- Time contributed by **"merging" time** (Asymptotically.)

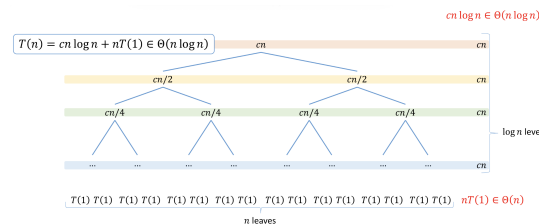
Comparing these contributions to the guess:

- If either is larger than the guess, try a **larger guess**.
- If both are smaller than the guess, try a **smaller guess**.
- If both are equal to the guess, add a  $\log n$  factor.

### 2.3 Recursion Tree

**Recursion tree** involves **visualising** the recursive calls as a tree, and then summing up the contributions from each level of recursion.

$$\text{e.g. } T(n) = \begin{cases} \Theta(1) & \text{if } n \leq 1 \\ 2T\left(\frac{n}{2}\right) + cn & \text{if } n > 1 \end{cases}$$



### 2.4 Master Theorem

**Master Theorem** provides a **closed-form solution** to recurrences of the form

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

where  $a \geq 1$  and  $b > 1$  are constants, and  $f(n) \in \Omega(1)$ . For  $d = \log_b a$ , and  $\epsilon > 0$ :

- $f(n) \in O(n^{d-\epsilon}) \implies T(n) \in \Theta(n^d).$
- $f(n) \in \Theta(n^d \log^k n)$  for some  $k \geq 0 \implies T(n) \in \Theta(n^d \log^{k+1} n).$
- $f(n) \in \Omega(n^{d+\epsilon})$ , and  $a f\left(\frac{n}{b}\right) \leq c f(n)$  for some constant  $c < 1 \implies T(n) \in \Theta(f(n)).$

Floors and ceilings can be ignored in the Master Theorem as they **do not affect the asymptotic behaviour** of the recurrence.

### Extended Master Theorem

Consider a recurrence of the form

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

where  $a \geq 1$  and  $b > 1$  are constants. For  $d = \log_b a$ , and  $f(n) \in \Theta(n^d \log^k n)$ :

- If  $k > -1$ , then  $T(n) \in \Theta(n^d \log^{k+1} n).$
- If  $k = -1$ , then  $T(n) \in \Theta(n^d \log \log n).$
- If  $k < -1$ , then  $T(n) \in \Theta(n^d).$

**Proof:** Consider  $T(n) = aT\left(\frac{n}{b}\right) + n^d \log^k n$ . Dividing both sides by  $n^d$ :

$$\frac{T(n)}{n^d} = \frac{T\left(\frac{n}{b}\right)}{\left(\frac{n}{b}\right)^d} + \log^k n$$

$$\frac{T\left(\frac{n}{b}\right)}{\left(\frac{n}{b}\right)^d} = \frac{T\left(\frac{n}{b^2}\right)}{\left(\frac{n}{b^2}\right)^d} + \log^k \left(\frac{n}{b}\right)$$

$$\frac{T\left(\frac{n}{b^2}\right)}{\left(\frac{n}{b^2}\right)^d} = \frac{T\left(\frac{n}{b^3}\right)}{\left(\frac{n}{b^3}\right)^d} + \log^k \left(\frac{n}{b^2}\right)$$

$\vdots$

$$\frac{T(n)}{n^d} = \log^k n + \log^k \left(\frac{n}{b}\right) + \log^k \left(\frac{n}{b^2}\right) + \dots + \Theta(1).$$

Assuming logarithms are base  $b$ :

$$\frac{T(n)}{n^d} = (\log n)^k + (\log n - 1)^k + (\log n - 2)^k + \dots + \Theta(1)$$

If  $k < -1$ , we have the  $p$ -series test with  $p = -k > 1$ , and thus:

$$\sum_{x=1}^{\log n} x^k = \Theta(1).$$

If  $k = -1$ , we have the harmonic series, and thus:

$$\sum_{x=1}^{\log n} x^{-1} = \Theta(\log \log n).$$

If  $k > -1$ , we have (sum of first  $n$  integers to the  $k$ -th power):

$$\sum_{x=1}^{\log n} x^k = \frac{\log^{k+1} n}{k+1} + O(\log^k n) = \Theta(\log^{k+1} n)$$

## 3 Proof of Correctness

### 3.1 Iterative Algorithms

- Loop Invariant**: A property that holds before each iteration of a loop.
- Initialization**: The invariant holds at the start of the first iteration.
- Maintenance**: If the invariant holds before an iteration, it holds after the iteration.
- Termination**: After the loop terminates, the invariant (and the termination condition) implies the correctness of the algorithm.

e.g. **Selection Sort**:

( $A[1..j-1]$  is sorted) and  
 $(\forall x \in A[1..j-1], y \in A[j..n], \quad x \leq y).$

### 3.2 Recursive Algorithms

- Base Case**: The algorithm works for the base case(s) (no recursive calls).
- Inductive Hypothesis**: Assume the algorithm is correct for all inputs of size less than  $n$ .
- Inductive Step**: Show that the algorithm works for an input of size  $n$ , using the inductive hypothesis.

## 4 Sorting

**Input**: An array  $A$  of  $n$  elements, and a total order  $\leq$  on the elements.

**Output**: A permutation of  $A$  such that  $A[1] \leq A[2] \leq \dots \leq A[n]$ .

## Properties of Sorting Algorithms

**Time complexity:** in the worst case, average case, or best case.

**Stability:** Original ordering is preserved for elements of equal values.

**In-place:** Very little extra space is used (e.g.  $O(1)$  or  $O(\log n)$ ).

### 4.1 Comparison-based Sorting

**Comparison-based sorting:** Sorting algorithms that only uses comparisons (and no other information) to determine the order of elements.

#### Lower bound for Comparison-based Sorting

**Claim:** The **worst-case time complexity** of any comparison-based sorting algorithm is  $\Omega(n \log n)$ .

**Proof:** Comparison-based sorting can be modelled as a **binary decision tree**. Each internal node represents a **comparison** between two elements, and each leaf represents a possible permutation of the input.

The worst-case running time  $\geq$  worst-case number of comparisons = height of the tree.

Since the **binary tree has at least  $n!$  leaves**, we have the height of the tree  $h \geq \log(n!)$ . Using Stirling's approximation, we have  $\log(n!) = \Theta(n \log n)$ .

### 4.2 Average-case Analysis for Quick Sort

If worst-case time complexities do not capture an algorithm's typical performance, we use **average-case analysis** for its expected time complexity.

**Claim:** The average-case time complexity of Quick Sort is  $O(n \log n)$ .

**Proof:** WLOG, assume all elements are distinct. Suppose  $a_1 < a_2 < \dots < a_n$ . The input array is a **random permutation  $\pi$**  of  $(a_1, a_2, \dots, a_n)$ . Suppose  $\pi$  is **uniformly random**. That is, each of the  $n!$  permutations is equally likely to be the input.

Then the pivot is equally likely to be any of the  $n$  elements, and the permutations for each of the recursive calls are also uniformly random. Then the **expected running time  $A(n)$**  is given by the recurrence:

$$\begin{aligned} A(n) &= \frac{1}{n} \sum_{j=1}^n [A(j-1) + A(n-j)] + cn \\ &= \frac{2}{n} \sum_{j=0}^{n-1} A(j) + cn \end{aligned}$$

Solving the recurrence, we have  $A(n) \in O(n \log n)$ .

## 5 Randomised Algorithms

### Freivalds' Algorithm

To verify if  $AB = C$  for  $n \times n$  matrices  $A, B, C$ , we can use Freivalds' algorithm:

1. Pick a random vector  $r \in \{0, 1\}^n$ .
2. Compute  $Br$ , then  $A(Br)$ , and  $Cr$ .
3. If  $A(Br) \neq Cr$ , return  $AB \neq C$ ; else, return  $AB = C$ .

**Principle of deferred decisions:**

$$\forall x, \quad \mathbb{P}\{E|X=x\} \geq p \implies \mathbb{P}\{E\} \geq p.$$